Unidad 5:

OPTIMIZACIÓN DE CONSULTAS E ÍNDICES EN SGBD

FICHA TÉCNICA DE LA UNIDAD DIDÁCTICA

Unidad de Trabajo	Unidad 5: Optimización de consultas e índices en MySQL / MariaDB
Módulo Profesional	Administración de Sistemas Gestores de Bases de Datos (ASGBD - Código: 0377)
Ciclo Formativo	2º Curso del Grado Superior en Administración de Sistemas Informáticos en Red (ASIR)
Resultados de Aprendizaje (RA)	**RA4:** Optimiza el rendimiento del sistema gestor monitorizando los accesos y ajustando los parámetros del motor.
Criterios de Evaluación (CE)	**CE 4.b:** Se han identificado los factores que afectan al rendimiento del sistema. **CE 4.c:** Se han analizado las trazas de ejecución de las consultas. **CE 4.d:** Se han creado índices y otras estructuras de optimización. **CE 4.f:** Se han reestructurado los datos para mejorar la velocidad de acceso.
Tiempo Estimado	10 horas lectivas

ÍNDICE DE CONTENIDOS

1. INTRODUCCIÓN

2. FUNDAMENTOS Y ESTRUCTURA DE LOS ÍNDICES

    2.1 La Regla del Equilibrio de Escritura-Lectura

    2.2 Tipos de Índices Lógicos

    2.3 Arquitectura Física de Almacenamiento: Árboles B

3. GESTIÓN AVANZADA DE ÍNDICES

3.1 Creación de Índices

3.2 Índices Compuestos y la Regla del Prefijo Izquierdo

3.3 Índices de Prefijo de Columna

3.4 Índices de Texto Completo (FULLTEXT)

3.5 Índices Funcionales

4. MANTENIMIENTO Y DIAGNÓSTICO DE ÍNDICES

4.1 Inspección del Estado de Índices

4.2 Reorganización, Desfragmentación y Estadísticas

5. PLANES DE EJECUCIÓN CON EXPLAIN

5.1 Estructura e Interpretación de Columnas Clave

6. CASOS PRÁCTICOS Y ESCENARIOS

6.1 Caso 1: Optimización de Búsquedas Simples

6.2 Caso 2: Pérdida de Sargabilidad por Uso de Funciones

6.3 Caso 3: Búsquedas de Patrones Intermedios (LIKE %text%) vs. FULLTEXT

1. INTRODUCCIÓN

En entornos corporativos y servidores de alta concurrencia, el tiempo de respuesta de las consultas SQL es un factor crítico de infraestructura. Para un Administrador de Sistemas (SysAdmin) o Administrador de Bases de Datos (DBA), el rendimiento del hardware del host (CPU, operaciones de I/O en disco, consumo de memoria RAM) está directamente acoplado a la eficiencia de las consultas ejecutadas sobre el motor de bases de datos.

La optimización de consultas (*Query Optimization*) busca minimizar el coste de ejecución del plan seleccionado por el SGBD. La herramienta de mayor impacto directo sobre este proceso es la creación e implementación estratégica de **índices**.

2. FUNDAMENTOS Y ESTRUCTURA DE LOS ÍNDICES

Un índice es una estructura de datos física auxiliar que el motor de almacenamiento (por ejemplo, *InnoDB* en MySQL) asocia a una tabla para acelerar la localización de registros sin necesidad de realizar barridos secuenciales completos de los archivos de datos en disco (*Full Table Scans*).

2.1 La Regla del Equilibrio de Escritura-Lectura

Aunque los índices aceleran exponencialmente las lecturas (SELECT), introducen un coste que el administrador debe balancear cuidadosamente:

- **Sobrecarga de Almacenamiento:** Cada índice requiere su propio espacio físico de almacenamiento en el tablespace (archivos .ibd).
- **Sobrecarga en Operaciones DML (INSERT, UPDATE, DELETE):** Cada vez que un registro muta, se inserta o se borra de una tabla, el motor de bases de datos debe reorganizar y reescribir de forma síncrona las estructuras de índices asociadas en disco, penalizando el rendimiento de escritura.
- **Complejidad del Optimizador:** Un exceso de índices incrementa el tiempo de computación que el optimizador de costes de MySQL requiere para evaluar y decidir el plan de ejecución más óptimo.

2.2 Tipos de Índices Lógicos

Los SGBD clasifican los índices lógicos en función de la naturaleza de los datos que indexan y de las restricciones semánticas aplicadas:

1. **Índices de Clave Primaria (*Primary Key*):** Identifican un registro de manera unívoca. No permiten valores duplicados ni nulos (NULL). En InnoDB, la Clave Primaria determina físicamente el orden en el que se almacenan las filas en disco (Índice Clustered o Agrupado).
2. **Índices de Clave Ajena (*Foreign Key*):** Indizan de forma automática las columnas que actúan como enlaces relacionales con otras tablas para optimizar el rendimiento de las operaciones JOIN y verificar la integridad referencial de forma ágil.
3. **Índices Únicos (*Unique Index*):** Garantizan que no existan registros repetidos en la columna indexada. A diferencia de las claves primarias, admiten valores nulos (NULL).
4. **Índices con Valores Repetidos (Índices Secundarios o No Únicos):** Optimizan las búsquedas de columnas no restrictivas (por ejemplo, apellidos, estados de pedidos o códigos de país).
5. **Índices de Columnas Múltiples (Compuestos):** Estructurados sobre una secuencia de varias columnas pertenecientes a la misma tabla.
6. **Índices de Texto Completo (*Fulltext*):** Especializados en la búsqueda léxica y semántica de grandes campos de caracteres de texto (VARCHAR, TEXT), permitiendo búsquedas rápidas mediante palabras clave.
7. **Índices Funcionales:** Permiten generar índices sobre el resultado de evaluar expresiones o funciones sobre los datos en lugar de los valores planos de la columna.

2.3 Arquitectura Física de Almacenamiento: Árboles B

En MySQL/MariaDB, la inmensa mayoría de los índices tradicionales se organizan físicamente bajo estructuras de **Árboles B** (específicamente *B+ Trees* en motores modernos como

InnoDB).

La selectividad de un índice se puede medir de forma matemática. La **Selectividad ($$$)** de un índice viene dada por la razón entre el número de valores únicos y el número total de registros de la tabla:

$$$ = \frac{\text{Cardinalidad de valores únicos (Distinct)}}{\text{Número total de filas de la tabla}}$$

Una selectividad óptima se aproxima a 1 (por ejemplo, en un índice de clave primaria o un campo NIF), lo que indica que el índice discriminará eficientemente las búsquedas. Una selectividad cercana a 0 (por ejemplo, indexar el campo *género* o *estado_activo*) indica que el índice apenas aportará ventajas y el optimizador de costes preferirá realizar un escaneo completo de la tabla.

Esquema Conceptual de B+ Tree en InnoDB:

En un árbol B+, los nodos internos contienen únicamente claves de indexación y punteros de navegación, mientras que la información física real (los registros de datos o los punteros de clave primaria) reside exclusivamente en los **nodos hoja**. Adicionalmente, todos los nodos hoja están doblemente enlazados entre sí, lo que facilita enormemente las consultas de rango y las ordenaciones secuenciales rápidas.

[Nodo Raíz]
/ \
[Nodo Interno] [Nodo Interno]
/ \ / \
[Hoja] <-> [Hoja] <-> [Hoja] <-> [Hoja]
(Datos) (Datos) (Datos) (Datos)

3. GESTIÓN AVANZADA DE ÍNDICES

3.1 Creación de Índices

MySQL permite declarar índices mediante tres mecanismos diferenciados.

Método 1: Sentencia Directa CREATE INDEX

Es la sentencia estándar e imperativa para crear índices secundarios en caliente sobre tablas ya existentes.

```
-- Crear un índice no único estándar
CREATE INDEX idx_pais ON cliente(pais);

-- Crear un índice único restrictivo
CREATE UNIQUE INDEX idx_email ON empleado(email);
```

Método 2: Sentencia de Alteración Estructural ALTER TABLE

Permite añadir índices utilizando la sintaxis de alteración física de la tabla. Con este método es posible omitir el nombre del índice de forma explícita; el motor le asignará de forma automática el nombre de la columna indexada.

```
-- Creación con nombre explícito
ALTER TABLE cliente ADD INDEX idx_nombre (nombre);

-- Creación sin especificar nombre (se llamará de forma implícita "nombre")
ALTER TABLE cliente ADD INDEX (nombre);
```

Método 3: Declaración inline en CREATE TABLE

Definición de índices físicos de manera concurrente en la estructura de creación inicial de la tabla.

```
CREATE TABLE cliente (
id INT UNSIGNED AUTO_INCREMENT,
nombre VARCHAR(50) NOT NULL,
email VARCHAR(150) NOT NULL,
telefono VARCHAR(9) NOT NULL,
PRIMARY KEY (id),
UNIQUE KEY uq_email (email),
INDEX idx_nombre (nombre)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

3.2 Índices Compuestos y la Regla del Prefijo Izquierdo

Un **Índice Compuesto** es aquel creado sobre más de una columna de forma conjunta. Su utilidad y explotación sigue de manera estricta la **Regla del Prefijo Izquierdo** (*Left-Prefix Rule*).

Ejemplo de Declaración:

```
CREATE INDEX idx_apellido_nombre ON cliente (apellido_contacto, nombre_contacto);
```

Este índice compuesto físico se almacena ordenado jerárquicamente: primero por apellido_contacto y, ante colisión de apellidos, se ordena internamente por nombre_contacto. Por tanto, el optimizador de MySQL solo podrá utilizar este índice en consultas que filtren:

1. `Por el apellido y nombre simultáneamente (WHERE apellido_contacto = 'Gómez' AND nombre_contacto = 'Ana').`
2. Únicamente por la columna del prefijo izquierdo (WHERE apellido_contacto = 'Gómez').

¿Qué ocurre si filtramos únicamente por nombre_contacto? El optimizador **no podrá utilizar el índice** y se verá obligado a realizar un escaneo completo de la tabla, ya que el árbol no está ordenado globalmente por nombres, sino por apellidos primero.

3.3 Índices de Prefijo de Columna

Para optimizar el tamaño de los índices y ahorrar consumo de memoria RAM en el buffer de índices, se puede limitar el árbol para que almacene únicamente los primeros caracteres de una columna de tipo cadena de texto de gran longitud.

```
-- Columna nombre_cliente definida originalmente como VARCHAR(50)
-- Creamos un índice que almacenará solo los primeros 25 caracteres
CREATE INDEX idx_nombre_cliente ON cliente (nombre_cliente(25));
```

Cálculo Metodológico de la Longitud Óptima del Prefijo:

Para determinar el número óptimo de caracteres necesarios para el prefijo de forma que no perdamos discriminación, debemos evaluar de forma analítica la ganancia de valores distintos agregando bytes.

```
-- 1. Evaluamos la cardinalidad total de valores únicos en la columna entera
SELECT COUNT(DISTINCT nombre_cliente) FROM cliente;

-- 2. Evaluamos la cardinalidad de valores únicos utilizando un prefijo de N caracteres
SELECT
COUNT(DISTINCT LEFT(nombre_cliente, 5)) AS pref_5,
COUNT(DISTINCT LEFT(nombre_cliente, 10)) AS pref_10,
COUNT(DISTINCT LEFT(nombre_cliente, 12)) AS pref_12,
COUNT(DISTINCT LEFT(nombre_cliente, 15)) AS pref_15
FROM cliente;
```

El DBA seleccionará el menor prefijo N que proporcione una cardinalidad de valores únicos muy cercana al 100% de la columna sin prefijo. De este modo, optimizamos el tamaño del archivo indexado en el disco sin introducir falsos positivos de búsqueda en el índice.

3.4 Índices de Texto Completo (FULLTEXT)

Los índices estándar basados en Árboles B no son eficientes para realizar búsquedas semánticas sobre grandes campos de texto desestructurado. Para resolver esto, MySQL proporciona los índices FULLTEXT en los motores InnoDB y MyISAM, implementando índices invertidos estructurados léxicamente.

Ejemplo de Declaración:

```
CREATE FULLTEXT INDEX idx_nombre_descripcion ON producto(nombre, descripcion);
```

Sintaxis de Consulta:

La explotación de estos índices se realiza utilizando las funciones específicas MATCH() y AGAINST().

```
SELECT nombre, descripcion
FROM producto
WHERE MATCH(nombre, descripcion) AGAINST ('acero' IN NATURAL LANGUAGE MODE);
```

Modos de Búsqueda Full-Text:

1. **IN NATURAL LANGUAGE MODE (Por defecto):** Trata los datos de entrada como una frase en lenguaje natural. No admite operadores lógicos ni caracteres de control especiales.

2. **IN BOOLEAN MODE:** Permite el uso de operadores booleanos avanzados para definir condiciones complejas:
    - + : Indica que la palabra **debe** aparecer de forma obligatoria.
    - - : Indica que la palabra **no debe** aparecer en el resultado.
    - * : Actúa como comodín de truncamiento a la derecha de la cadena (por ejemplo, fruta* busca "fruta", "frutas", "frutales").
    - " " : Delimita una frase literal que debe coincidir de forma secuencial exacta.

Ejemplo en Modo Booleano:

```
-- Buscar filas con la palabra "planta" de forma obligatoria, pero que NO contengan la palabra "árbol"
SELECT * FROM producto
WHERE MATCH(nombre, descripcion) AGAINST ('+planta -arbol' IN BOOLEAN MODE);
```

3.5 Índices Funcionales

A partir de la versión **8.0.13 de MySQL**, es posible generar índices físicos sobre el resultado devuelto de expresiones, funciones de manipulación de cadenas de caracteres u operaciones matemáticas.

Ejemplo de Declaración:

```
-- Crear un índice sobre la expresión del año de la columna fecha_pago
CREATE INDEX idx_year_functional_index ON pago ((YEAR(fecha_pago)));
```

4. MANTENIMIENTO Y DIAGNÓSTICO DE ÍNDICES

4.1 Inspección del Estado de Índices

Un SysAdmin debe auditar con frecuencia el estado físico de sus tablas e índices para evaluar si es preciso reconstruirlos.

Comando SHOW INDEX:

```
SHOW INDEX FROM cliente FROM jardineria;
```

Atributos Clave devueltos en SHOW INDEX:

- **Table:** El nombre físico de la tabla asociada.
- **Non_unique:** Si devuelve 0, indica que es un índice único (no permite duplicidad física). Si es 1, permite valores duplicados.
- **Key_name:** Nombre del índice.

- **Seq_in_index:** Orden posicional de la columna actual dentro de un índice compuesto (comienza en 1).
- **Column_name:** Nombre físico del campo indexado.
- **Collation:** Ordenamiento del índice (en árboles B suele ser A para orden ascendente, o NULL si no hay ordenamiento).
- **Cardinality:** Una estimación de la cantidad de valores únicos almacenados en el índice. A mayor número de cardinalidad respecto al número de filas de la tabla, mayor selectividad tendrá.
- **Index_type:** El método de almacenamiento físico utilizado (por ejemplo, BTREE, FULLTEXT).

También es viable realizar diagnósticos rápidos utilizando DESCRIBE o SHOW COLUMNS:

DESCRIBE cliente;

La columna Key mostrará etiquetas descriptivas como PRI (Primary Key), UNI (Unique Index), o MUL (índice no único con valores repetidos o columna prefijo de índice compuesto).

4.2 Reorganización, Desfragmentación y Estadísticas

Con la actividad de inserción y borrado, los índices sufren degradación de rendimiento por fragmentación física. El motor dispone de dos comandos de mantenimiento crítico:

Comando ANALYZE TABLE:

Analiza y almacena en el diccionario de datos del SGBD la distribución exacta de claves de la tabla. El optimizador utiliza estas estadísticas para calcular los costes de los planes de consulta y decidir el orden óptimo de los JOIN. Se aconseja ejecutarlo tras cargas masivas de datos.

ANALYZE TABLE cliente, pago;

Comando OPTIMIZE TABLE:

Realiza una desfragmentación de las tablas de datos e índices, liberando el espacio vacío que ha quedado reservado por operaciones de borrado previas, y reordenando físicamente las claves en disco.

OPTIMIZE TABLE cliente;

5. PLANES DE EJECUCIÓN CON EXPLAIN

EXPLAIN es la instrucción diagnóstica que nos permite analizar de qué manera el motor de almacenamiento de bases de datos va a resolver una consulta SQL antes de ejecutarla, desglosando los costes previstos.

EXPLAIN SELECT nombre_contacto, telefono FROM cliente WHERE pais = 'France';

5.1 Estructura e Interpretación de Columnas Clave

Al ejecutar EXPLAIN, el SGBD devuelve una fila por cada tabla implicada en la consulta con la siguiente estructura de columnas:

```
Columna
Significado Técnico e Impacto en el
Rendimiento


id
Secuenciador identificativo del identificador de la consulta del
select.


select_type
El tipo de consulta analizada (por ejemplo: SIMPLE para consultas
sin subconsultas ni uniones, PRIMARY para el SELECT principal en
subconsultas, SUBQUERY).


table
La tabla física del disco sobre la cual se está procesando la
información.


type
La columna más crítica del EXPLAIN. Indica el
método de acceso que el motor va a realizar para recuperar las filas. Se
evalúa de mejor a peor rendimiento. (Ver detalle a continuación).


possible_keys
Los nombres de los índices de la tabla que el optimizador podría
haber utilizado para recuperar los datos.


key
El índice que el optimizador ha seleccionado finalmente para
ejecutar la consulta. Si es NULL, indica que MySQL no está utilizando
ningún índice.


key_len
La longitud en bytes de la clave del índice seleccionada por el
```

```
motor. Útil para determinar cuántas columnas de un índice compuesto se
están utilizando realmente.


ref
Muestra qué columnas o constantes se comparan con el índice
especificado en la columna key.


rows
El número aproximado de filas físicas que el motor estima que tendrá
que leer y analizar en disco para retornar el conjunto de datos de la
consulta.


filtered
Porcentaje estimado de filas filtradas que serán evaluadas por el
motor antes de aplicar las condiciones del WHERE sobre el total de filas
indicadas en rows.


Extra
Información adicional sobre cómo procesa MySQL la consulta (por
ejemplo: Using index indica que la consulta se resuelve íntegramente
leyendo el índice sin consultar la tabla física; Using filesort indica
un coste de CPU muy alto debido a ordenaciones en memoria sin índices;
Using temporary indica la creación de tablas temporales internas).
```

Valores de la Columna type (Ordenados de mejor a peor rendimiento):

1. **system:** La tabla tiene una única fila (caso excepcional).
2. **const:** La tabla se lee como máximo una vez al inicio de la consulta por coincidencia exacta con una clave primaria o un índice único (UNIQUE). Es el acceso de tiempo constante óptimo.
3. **eq_ref:** Se utiliza un índice único o clave primaria para resolver un JOIN con la tabla anterior. Se accede de forma unívoca a un registro por cada fila procesada de la primera tabla.
4. **ref:** Se realiza una búsqueda por índice secundario no único utilizando un operador de coincidencia exacta (=).
5. **range:** Se realiza un escaneo de un rango ordenado de claves de un índice, resolviendo condiciones que usan operadores como >, <, BETWEEN, IN(), o LIKE 'prefix%'.

6. **index:** Se escanea el árbol de índices de forma completa en lugar de la tabla física (evita I/O de disco pesado de columnas no indexadas, pero requiere procesar el árbol entero).
7. **ALL: Full Table Scan.** El motor realiza un escaneo secuencial de todas las páginas de datos físicas de la tabla en disco. Extremadamente ineficiente en tablas medianas y grandes, provocando un elevado consumo de CPU y disco.

6. CASOS PRÁCTICOS Y ESCENARIOS

6.1 Caso 1: Optimización de Búsquedas Simples

Consulta Original:

SELECT nombre_contacto, telefono FROM cliente WHERE pais = 'France';

Análisis Inicial con EXPLAIN:

EXPLAIN SELECT nombre_contacto, telefono FROM cliente WHERE pais = 'France';

Resultado del Plan:

- type = ALL
- possible_keys = NULL
- key = NULL
- rows = 36 (Número total de filas de la tabla cliente)
- Extra = Using where

Diagnóstico: Como no existe un índice físico sobre el campo pais, el motor de base de datos se ve obligado a realizar un escaneo completo de toda la tabla para comprobar el valor del país en cada una de las 36 filas de datos.

Corrección y Optimización:

```
-- Creamos un índice secundario B-Tree sobre la columna de filtrado
CREATE INDEX idx_pais ON cliente(pais);
```

Análisis Posterior con EXPLAIN:

EXPLAIN SELECT nombre_contacto, telefono FROM cliente WHERE pais = 'France';

Resultado del Plan Optimizado:

- type = ref
- possible_keys = idx_pais
- key = idx_pais

- rows = 2
- Extra = NULL

Diagnóstico: Tras la creación de la estructura del índice, el motor accede directamente a las direcciones físicas de los registros utilizando el árbol idx_pais. El tiempo de respuesta es óptimo: ya no requiere un escaneo completo en disco, limitando la lectura a las únicas 2 filas estimadas que cumplen la condición de búsqueda.

6.2 Caso 2: Pérdida de Sargabilidad por Uso de Funciones

Un fallo de diseño muy recurrente en administración de bases de datos es escribir consultas que utilicen funciones matemáticas, de manipulación de cadenas de texto o de fecha directamente sobre la columna de filtrado bajo un bloque WHERE. Esto inhabilita la capacidad del motor de aprovechar un índice tradicional (la consulta pierde su **Sargabilidad** - *Search Argument Able*).

Escenario de Comparación:

Opción A: Consulta usando la función YEAR sobre la columna fecha_pago

```
SELECT AVG(total) FROM pago WHERE YEAR(fecha_pago) = 2008;
```

Opción B: Consulta usando condiciones de rango sobre la columna fecha_pago

```
SELECT AVG(total) FROM pago
WHERE fecha_pago >= '2008-01-01' AND fecha_pago <= '2008-12-31';
```

Evaluación de Rendimiento con EXPLAIN (asumiendo que existe un índice sobre fecha_pago):

- Al analizar la **Opción A**, el motor devuelve type = ALL. Esto sucede porque, para evaluar el año de cada fecha de pago, el motor debe extraer de forma secuencial cada registro físico de la tabla en disco, pasar el valor de la fecha por la función YEAR() en la CPU del host y comprobar si el resultado es igual a 2008. Al aplicar la función a nivel de columna, **el SGBD inhabilita el árbol B-Tree**, forzando un escaneo completo.
- Al analizar la **Opción B**, el motor devuelve type = range y selecciona el índice idx_fecha_pago. El motor puede navegar directamente por la estructura del índice de forma sargable, identificando el inicio del rango ('2008-01-01') y recuperando de forma consecutiva las claves de forma secuencial hasta llegar al fin del rango ('2008-12-31').

> 💡 **Nota de Ingeniería de Rendimiento (DBA):** En MySQL 8.0, si el uso de la función YEAR() es un requisito obligatorio del desarrollo, el SysAdmin puede forzar al optimizador a utilizar el índice mediante un **Índice Funcional** (CREATE INDEX idx ON pago ((YEAR(fecha_pago)))). No obstante, en términos generales de portabilidad y buenas prácticas, siempre es preferible reescribir las consultas en formato de rangos numéricos puros para mantener la sargabilidad estándar del sistema.

6.3 Caso 3: Búsquedas de Patrones Intermedios (LIKE %text%) vs. FULLTEXT

Consulta Ineficiente con Patrón Intermedio:

SELECT * FROM producto WHERE nombre LIKE '%acero%' OR descripcion LIKE '%acero%';

Análisis con EXPLAIN:

EXPLAIN SELECT * FROM producto WHERE nombre LIKE '%acero%' OR descripcion LIKE '%acero%';

Resultado del Plan:

- type = ALL
- rows = 276 (Barrido secuencial completo de la tabla de productos)

¿Por qué un índice estándar B-Tree no sirve en un LIKE %text%? Los árboles B ordenan y asocian las cadenas por sus prefijos. Si el patrón de búsqueda comienza por el carácter comodín % (por ejemplo, %acero), el motor de bases de datos no puede determinar por qué nodo o rama del árbol empezar a navegar, ya que no conoce los caracteres iniciales de la palabra, viéndose obligado a descartar el índice y realizar un escaneo completo (type = ALL).

Solución de Optimización mediante Índices de Texto Completo:

```
-- 1. Creamos el índice físico invertido de texto completo
CREATE FULLTEXT INDEX idx_nombre_descripcion ON producto (nombre, descripcion);

-- 2. Reescribimos la consulta utilizando la sintaxis MATCH / AGAINST
SELECT * FROM producto
WHERE MATCH(nombre, descripcion) AGAINST ('acero');
```

Análisis con EXPLAIN del Plan Optimizado:

EXPLAIN SELECT * FROM producto WHERE MATCH(nombre, descripcion) AGAINST ('acero');

Resultado del Plan:

- type = fulltext
- possible_keys = idx_nombre_descripcion
- key = idx_nombre_descripcion
- rows = 1 (MySQL lee directamente la lista invertida correspondiente al token 'acero' en un solo paso)